

JavaScript Learning Roadmap

From Basics to Advanced • 27 Topics • 25 Interview Questions

Stage 1 Get the Basics Right

7 Topics

Variables, Functions, Arrays, Objects...

Stage 2 Understand How JS Works

9 Topics

Closures, Promises, Event Loop, Modules...

Stage 3 Advance

11 Topics

Currying, Proxy, Design Patterns, Polyfills...

What's Inside

27

Core JS Topics

25

Interview Questions

3

Learning Resources

Master JavaScript — one concept at a time.

Get the Basics Right

01

Data Types & typeof

Primitives vs reference types, typeof quirks, Symbol, BigInt

02

var / let / const + Scope

Block scope, function scope, hoisting differences, TDZ intro

03

Type Coercion & == vs ===

Implicit vs explicit coercion, truthy/falsy, abstract equality

04

Functions

Declaration, expression, arrow — and how they differ in hoisting & this

05

Arrays & common methods

map, filter, reduce, find, some, every, splice vs slice

06

Objects & destructuring

Dot vs bracket, computed keys, nested destructuring, defaults

07

Operators — ??, ?., spread, rest

Nullish coalescing, optional chaining, rest params, spread syntax

PRO TIP

Don't rush through basics. Build 2-3 mini projects (calculator, to-do list, quiz app) using only vanilla JS before moving to Stage 2.

Understand How JS Actually Works

08

Closures & Lexical Scope

How inner functions remember outer variables, practical use cases

09

The this keyword (all 4 rules)

Default, implicit, explicit, new binding — and arrow function exception

10

call / apply / bind

Function borrowing, partial application, explicit context setting

11

Prototype Chain & Inheritance

How JS objects inherit, `__proto__` vs prototype, lookup chain

12

ES6 Classes

Syntactic sugar over prototypes, constructor, extends, super

13

Event Loop

Call stack, task queue, microtask queue — how async works under the hood

14

Promises

States (pending/fulfilled/rejected), chaining, `.all/.race/.any/.allSettled`

15

async / await + error handling

Syntactic sugar over promises, try-catch patterns, sequential vs parallel

16

ES Modules — import/export

Named vs default exports, dynamic `import()`, tree shaking basics

17

Execution Context & Call Stack

Global/function execution context, creation & execution phase

18

Hoisting in depth + TDZ

var vs let/const hoisting, Temporal Dead Zone, function hoisting

19

Currying & Partial Application

Transform $f(a,b,c)$ into $f(a)(b)(c)$, real-world use cases

20

Memoization

Cache expensive function results, fibonacci example, when to use

21

Debounce & Throttle

Rate-limiting functions, search input optimization, scroll handlers

22

Iterators & Generators

Symbol.iterator, function*, yield, lazy evaluation, async generators

23

Proxy & Reflect

Meta-programming, traps, validation wrappers, reactive systems

24

WeakMap / WeakSet / WeakRef

Garbage collection, private data patterns, memory management

25

Error Handling patterns

Custom errors, error boundaries, graceful degradation strategies

26

Design Patterns in JS

Singleton, Observer, Factory, Module, Strategy — when to use each

27

Writing Polyfills from scratch

map, bind, Promise.all — understanding internals by building them

Learning Resources

Namaste JavaScript

PLAYLIST



Ep.1 — How JS Works

26 Episodes

Namaste JavaScript

Akshay Saini

Best for Stage 2 & 3

JS internals — closures, event loop, promises. The gold standard for understanding how JavaScript really works under the hood.

youtube.com/playlist?list=PLlasXeu85E9cQ32gLCvAvr9vNaUccPVNP

Learn JavaScript

VIDEO



Full Course for Beginners

7 Hours

JavaScript Full Course

freeCodeCamp

Best for Stage 1 & 2

7 hours, beginner to advanced. A comprehensive single-video course covering fundamentals through intermediate concepts.

youtube.com/watch?v=jS4aFq5-91M

JavaScript Tutorials

PLAYLIST



#1 — Introduction

51 Videos

JavaScript Tutorials

The Net Ninja

Follow topic by topic

Short, focused videos organized by topic. Perfect for following along with this roadmap one concept at a time.

youtube.com/playlist?list=PL4cUxeGkcC9haFPT7J25Q9GRB_ZkFrqAV

Suggested Study Plan

- **Week 1-2** Stage 1 — Basics (1 topic/day + mini project)
- **Week 3-4** Stage 2 — JS Internals (watch Namaste JS alongside)
- **Week 5-6** Stage 3 — Advanced (build & implement each concept)
- **Week 7-8** Interview Questions — practice all 25, write code daily

25 JS Interview Questions

Part 1: Closures & Scope • this & Functions • Async & Event Loop

Closures & Scope

Q1 What will this print and why?

Q2 How do you create private variables in JS without classes?

Q3 What is the difference between lexical scope and dynamic scope?

JavaScript

```
for (var i = 0; i < 3; i++) {  
  setTimeout(() => console.log(i), 0)  
}
```

this & Functions

Q4 What is the output of this code? (see code block below)

Q5 How does bind work? Write it from scratch.

Q6 What happens when you use new with an arrow function?

JavaScript

```
const obj = {  
  name: 'JS',  
  getName: () => console.log(this.name)  
}  
obj.getName()
```

Async & Event Loop

Q7 What is the output order and why? (see code block below)

Q8 How do you run 5 API calls in parallel and wait for all?

Q9 What happens if one Promise rejects inside Promise.all?

Q10 How do you handle errors in async/await properly?

JavaScript

```
console.log('1');  
console.log('4');  
Promise.resolve().then(() => console.log('3'));  
setTimeout(() => console.log('2'), 0);
```

25 JS Interview Questions

Part 2: Prototypes • Arrays & Objects • Hoisting • Coding

Prototypes & Classes

- Q11** What is the difference between `__proto__` and `prototype`?
- Q12** How does `Object.create()` work? When would you use it over `class`?
- Q13** What does `hasOwnProperty` do and when does it matter?

Arrays & Objects

- Q14** What is the difference between `map` and `forEach`?
- Q15** Write a `deepClone` function without `JSON.stringify`.
- Q16** Shallow copy vs deep copy? When does spread `{...obj}` fail?

Hoisting & Execution

- Q17** Function declaration vs expression — hoisting difference?
- Q18** What will `console.log(typeof x); let x = 5` output?
- Q19** Order of function declarations vs variable declarations?

Practical / Coding Challenges

- Q20** Write a `debounce` function from scratch.
- Q21** Write your own `Array.prototype.map` polyfill.
- Q22** Write a `Promise.all` polyfill from scratch.
- Q23** Write a basic `Promise` polyfill — `resolve`, `reject`, `.then`.
- Q24** Flatten a deeply nested array without `.flat()`.
- Q25** Implement pipe: `pipe(f, g, h)(x) === h(g(f(x)))`

BONUS Q: Given a click handler — identify and fix the memory leak. (See full doc)

Happy Learning!

Master JavaScript — one concept at a time.

Follow for more JS content

[@debugging_life.js](#)

27

Topics

25

Questions

3

Stages

1

Roadmap

Save this PDF • Share with a friend • Start coding today